系统设计速成 · 替代 Alex Xu，一晚上能看完

带虚线下划线的名词点一下就地展开详细解释，再点收起；PDF 版在最后的「名词详解」附录。给 11–12 点 Kyle 那一小时（写代码 + 系统设计）。每个概念：先比方，再要点，再一个数字，最后一句英文。黄底「你的项目里」= 答题时拿来当例子的东西。不重复代码对照页和补充手册已有的内容。

0 · 系统设计题怎么答（4 步，30 分钟分法）

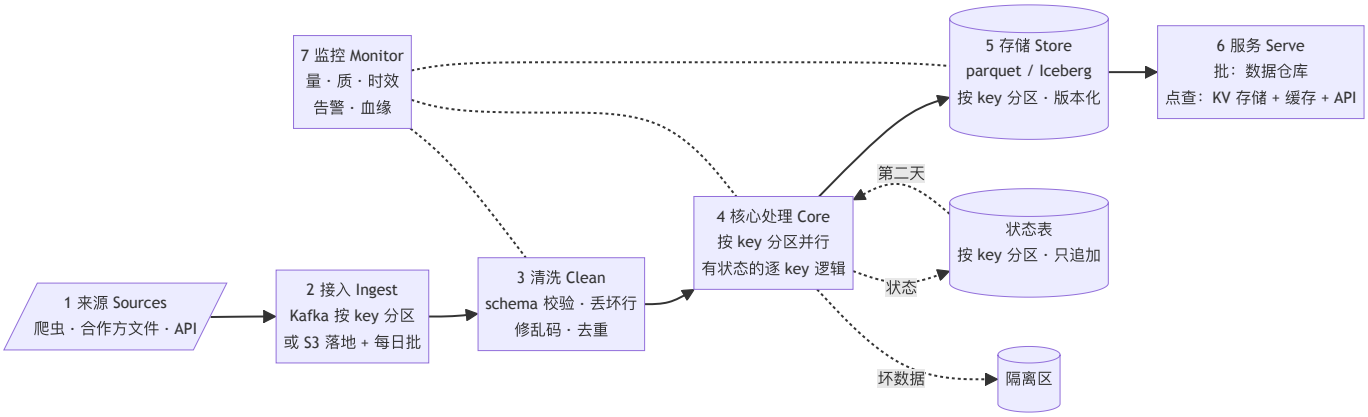
- **第 1 步 · 问清楚要求（3–5 分钟）（clarify requirements）**：功能要什么、不要什么；规模多大（用户数、每天多少数据、读多写多）；延迟要求（秒级还是天级）；一致性要求（能不能晚一点看到）。先问再画，别一上来就画框。
- **第 2 步 · 估算（3 分钟）（back-of-envelope）**：QPS、存储、带宽，算到数量级就行（见第 1 节）。
- **第 3 步 · 画高层图（10 分钟）（high-level design）**：数据从哪来 → 经过什么 → 存哪 → 谁来读。每个框一句话说为什么要它。
- **第 4 步 · 深挖 1–2 个点（10 分钟）（deep dive）**：他最关心的那个瓶颈。说清取舍：选了 A 放弃了 B，因为 C。
- 最后 2 分钟：瓶颈在哪、怎么扩、怎么监控、下一步。

Before I draw anything, can I check a few things: what's the daily volume, how fresh does the result need to be, and who reads it? Then I'll estimate the numbers, sketch the pipeline, and go deep on the part you care most about.

开口模板。他们是数据公司，题目八成是「数据流水线」而不是「设计 Twitter」。

万能模板图（答不出来就画这个，再往里填）

任何「数据流水线」题都是这六个框。先把框画出来，再一个一个说每个框里用什么、为什么。



照着说的顺序（每框一句）：

- 1 来源：数据从哪来、每天多少、格式是什么、会不会迟到。
- 2 接入：要秒级就 Kafka（按 user_id 分区保序），要天级就 S3 落文件每天跑一批。
- 3 清洗：schema 契约、丢坏行不丢整批、修乱码、按内容哈希去重。
- 4 核心：按 key 分区让同一个人的数据在一台机器上；逐 key 的有状态逻辑（这就是你的状态机）；一个 key 出错隔离不阻塞。
- 5 存储：列式 parquet；表格式给事务、分区裁剪、时间旅行；状态按 key 分区只追加。
- 6 服务：分析查询走仓库；点查走 KV + 缓存 + API。
- 7 监控：每次运行出指标，突变告警，必须为 0 的硬断言。
- 最后说三句：瓶颈在 4（算力线性加机器）和 5（小文件）；扩容靠分区数；取舍是 X 换 Y 因为 Z。

Any data pipeline is the same six boxes: sources, ingest, clean, core processing partitioned by key, storage, and serving, with monitoring across all of them and a state store feeding the next run. Let me draw that and then fill in each box for this problem.

1・估算（back-of-envelope）

比方：装修前先量房间，不用精确到毫米，知道是 50 平还是 500 平就够。

记住这几个数	用法
1 天 ≈ 86,400 秒 ≈ **10 万秒**	每天 1 亿请求 ÷ 10 万秒 ≈ 1,000 QPS
1 KB × 10 亿 = 1 TB	每行 1 KB、10 亿行 = 1 TB
一台机器：内存 64–256 GB，SSD 几 TB，网卡 1–10 Gbps	超过就要分机器
内存读 100 ns、SSD 读 100 μs、跨机房网络 100 ms	每差 1000 倍；能放内存就别读盘
一个 CPU 核每秒能处理多少行：看每行干多少活（见下）	你实测 3.5 万行/核秒（含 Python 模糊匹配）

「一个核每秒处理几万到几十万行」怎么来的：一个 CPU 核每秒大约做 10^9 次简单操作。处理一行要做多少次操作，决定每秒能过多少行：

每行干的活	每行大约几次操作	每核每秒多少行	例子
纯向量化：过滤、加列、类型转换（Polars/Spark 的列式计算）	10^2–10^3	10^6–10^7（百万到千万）	你的 conform、clean、分桶
一般逐行逻辑：解析、几次比较、写出	10^4–10^5	10^4–10^5（几万到几十万）	普通 ETL 的估算默认值
重逻辑：Python 循环 + 字符串相似度 + 多候选打分	10^4–10^5，但 Python 本身慢 10–100 倍	10^4 左右	你的 step + match：3.5 × 10^4 行/核秒（指纹跳过一半帮了大忙）
每行调一次模型 / LLM	10^8 以上，或网络往返	10–100	这就是为什么不能 LLM 逐对判

用法：估算时先判断「每行是哪一档活」，再乘以行数得核秒。被问「怎么知道 3.5 万」：实测，1150 万行 ÷ (78 秒 × 4 核) ≈ 3.7 万，取 3.5 万保守。

A core does about a billion simple operations a second. Vectorised column work is a few hundred operations per row, so millions of rows per second; row-by-row logic in Python with fuzzy matching is tens of thousands per second, which is what I measured, 35 thousand rows per core-second. Calling a model per row drops that to tens, which is why an LLM can't sit on the hot path.

你的项目里：1000 亿行 × 约 500 字节 ≈ 50 TB 输入；3.5 万行/核秒 → 1000 亿 ÷ 3.5 万 ≈ 286 万核秒 ≈ 800 核时；64 核约 12 小时。输出约 1.08 倍输入。

At roughly 500 bytes a row, 100 billion rows is about 50 terabytes. At 35 thousand rows per core-second that's about 800 core-hours, so 12 hours on 64 cores or a bit over an hour on 640.

数量级换算表（10 的几次方）

10 的几次方	叫法	字节单位	直观
10^3	千 thousand（K）	1 KB ≈ 一条 1000 字的记录	一个人一段经历的所有字段约 0.5 KB
10^6	百万 million（M）	1 MB ≈ 一张照片	一百万行 × 1 KB = 1 GB
10^9	十亿 billion（G / B）	1 GB ≈ 一部电影	十亿行 × 1 KB = 1 TB
10^{12}	万亿 trillion（T）	1 TB ≈ 一块硬盘	1000 亿行 × 0.5 KB = 50 TB
10^{15}	千万亿 quadrillion（P）	1 PB = 1000 块硬盘	大公司数据湖级别

中文和英文的坑：中文按万进位，英文按千进位。

中文	数字	英文
1 万	10^4	10 thousand
10 万	10^5	100 thousand
100 万	10^6	1 million
1000 万	10^7	10 million
1 亿	10^8	100 million
10 亿	10^9	1 billion
1000 亿	10^{11}	100 billion
1 万亿	10^{12}	1 trillion

时间：1 天 = 86,400 秒 ≈ 10^5（算 QPS 时直接除 10 万）；1 月 ≈ 2.6×10^6 秒；1 年 ≈ 3.15×10^7 秒。

延迟（相差 1000 倍一档）：内存 100 ns = 10^{-7} 秒 → SSD 100 μs = 10^{-4} 秒 → 跨机房网络 100 ms = 10^{-1} 秒。

你的题目用这张表算一遍：

- 输入 11.5M 行 = 1.15×10^7；150k 人 = 1.5×10^5；每人约 77 行。
- 1000 亿行 = 10^{11}；× 500 B（5×10^2）= 5×10^{13} B = 50 TB。
- 3.5 万行/核秒 = 3.5×10^4；$10^{11} \div 3.5 \times 10^4 \approx 2.9 \times 10^6$ 核秒；÷ 3600 ≈ 800 核时。
- 64 核：800 ÷ 64 = 12.5 小时；640 核：1.25 小时。
- 每天 3 亿新行 = 3×10^8；$\div 3.5 \times 10^4 \approx 8.6 \times 10^3$ 核秒 ≈ 2.4 核时；4 核约 36 分钟。
- 1 万 QPS 的点查：$\times 10^5$ 秒/天 = 10^9 次/天。

I round everything to powers of ten: a day is ten to the fifth seconds, a kilobyte times a billion rows is a terabyte, and memory, SSD and network are each a thousand times apart. 100 billion rows at 500 bytes is 50 terabytes; at 35 thousand rows per core–second that's about 800 core–hours.

2 · 分区 / 分片（partitioning / sharding） + 一致性哈希

比方：一个图书馆放不下，按书名首字母分到 26 个馆。分区就是「按什么键把数据切开放到不同机器」。

- **按键取模（modulo / hash partitioning）**（你的 user_id % 64）：简单、均匀。缺点：桶数一变，几乎所有数据都要搬家（100 % 64 = 36，100 % 65 = 35）。
- **按范围（range partitioning）**：按数值区间切，user_id 1 到 100 万放机器 1，100 万到 200 万放机器 2，以此类推（像图书馆按编号分楼层）。好处：范围查询只问一台。坏处：容易热点，新注册的 id 都是最大的，全挤在最后一台。你的场景不查范围、只按人独立处理，所以用取模让它均匀。
- **一致性哈希（consistent hashing），先说它解决什么问题**：取模分区有个大麻烦。原来 4 台机器，数据按 id % 4 放。现在加 1 台变 5 台，变成 id % 5。id = 7 原来在 3 号机（7 % 4 = 3），现在要去 2 号机（7 % 5 = 2）。你去算会发现，几乎所有数据都要换机器。加一台机器要把整个库搬一遍，太贵。
- **一致性哈希怎么做**：比方：一条环形跑道，上面标着 0 到 100 的刻度。每台机器站在跑道上某个位置，比如 A 站 10、B 站 40、C 站 70。每条数据算一个哈希值，也是 0 到 100 之间的一个数，落在跑道上。规则只有一条：数据归「顺时针往前走遇到的第一台机器」。哈希值 25 的数据往前走遇到 B，归 B；哈希值 80 的往前走绕过 100 回到 10，遇到 A，归 A。
- **加机器只搬一小段**：新来一台 D 站在 55。原来 41 到 70 之间的数据都归 C，现在 41 到 55 这段归 D，56 到 70 还归 C。只有 C 的一部分数据搬给了 D，A 和 B 的数据一条都没动。有 N 台机器，加一台大约只搬 1/N 的数据，而不是全部。
- **虚拟节点（virtual nodes）**：上面 3 台机器站 10、40、70，看起来还算匀。可要是站成 10、12、14，那 14 到 100 这一大段全归 A，A 累死别人闲着。办法：每台机器不站一个点，站 100 到 200 个点，散在跑道各处。这样每台机器管的总长度就差不多。机器性能强的多站几个点，自然多管一点。
- **热点键（hot key）**：比方：一个超市 100 个收银台，但所有人都挤在卖茅台的那一个。某个 key 的流量特别大（一个大 V 的主页），它落在哪台机器哪台就被压垮。两个办法：① 把这个 key 拆成「key-1、key-2 … key-10」十份，分到十台机器，读的时候十份合起来；② 这个 key 单独给一台专用机器。
- **二级索引（secondary index）**：比方：通讯录按姓名排序，想按电话号码找人就得从头翻到尾。你按 user_id 分区，「查某个人」很快，直接去那一台。但「查所

有在 Google 工作过的人」，每台机器上都可能有，只能每台都问一遍再把结果拼起来（叫 scatter-gather，撒出去再收回来），慢。要想按公司查也快，就另外维护一份按公司名分区的数据（二级索引），写的时候两份都要更新。

你的项目里：取模是对的，因为状态按桶存、同一个人必须回到同一桶，而且桶数你自己定、_meta.json 守着。被问「桶数要变怎么办」：用 history 全量重跑一次换桶数；生产上要平滑扩容就上一致性哈希或者表格式自带的分区。Spark 版 4096 分区；按 user_id 分区让 join 状态表时同一个人在同一台机器上，不用网络搬数据。

I partition by user_id mod 64 because state is stored per bucket and the same user must land in the same bucket next time. Modulo doesn't rebalance gracefully, so for elastic scaling I'd use consistent hashing with virtual nodes, or let the table format own the partitioning.

3 · 复制与一致性

比方：一份账本抄三份放三个地方，一处烧了别处还在。复制 = 同一份数据存多份。

- **主从（leader–follower）**：写只去主，从库跟着抄；读可以分给从库。抄有延迟 → 刚写完去从库读可能读不到（**复制延迟（replication lag）**）。
- **强一致（strong consistency）**：写完所有人立刻看到同样的值。**最终一致（eventual consistency）**：一会儿之后大家才一样。数据流水线的输出通常接受最终一致（按天/按小时更新）。
- **CAP 三个字母**：C = Consistency 一致性，所有机器上读到的值一样。A = Availability 可用性，任何时候来请求都有回应。P = Partition tolerance 分区容忍，网络断成两半时系统还能工作。
- **P 是什么**：比方：两家分店靠电话对账，电话线断了，两家店互相不知道对方卖了什么。这就是「网络分区」，机器都活着，只是彼此联系不上。分布式系统里电话线一定会偶尔断，所以 P 不是你选不选的，是必须扛住的。
- **所以 CAP 实际是二选一**：电话断了的那段时间，分店只能二选一。要么停止卖货等电话恢复（保 C 丢 A：宁可不服务也不让两边账不一样）；要么照常卖、各记各的账，之后再对（保 A 丢 C：一直能用，但这段时间两边读到的值可能不同）。
- **你的场景**：每天一批、输出一次提交，读的人要么看昨天的完整版、要么看今天的完整版，不会读到一半。这是偏 C 的做法，代价是新数据要等到下一次提交才可见，天级新鲜度本来就允许。
- **英文**：P is partition tolerance: the network will split sometimes, so you have to tolerate it. During a split you choose: stay consistent and refuse some requests, or stay available and risk serving stale values. A daily batch with one atomic commit per run is on the consistent side, which the freshness requirement allows.
- **多副本写冲突（write conflict）**：两边同时改同一条 → 要定规则：最后写入赢（last-write-wins）（丢数据）、或者按版本号合并。

你的项目里：输出 parquet 放对象存储（S3 自带多副本），不用自己管复制；每天一版、最终一致。状态表用 Iceberg 的事务提交，读的人永远看到完整的一版，不会读到写一半。

For a daily batch output, eventual consistency is fine: readers see yesterday's complete version until today's commit lands atomically. I'd keep the data in object storage, which already replicates, and use a table format for atomic commits.

4 · 唯一 ID（Snowflake）

比方：车牌 = 省份 + 城市 + 序号，各段各管各的，不用全国排队领号。

- **Snowflake**：64 位 = 1 位符号 + 41 位时间戳（毫秒）+ 10 位机器号 + 12 位每毫秒序号。每台机器自己发号，不用中心协调；按时间大致有序。
- **为什么不用自增 ID（auto-increment）**：要一个中心数据库发号，所有机器都来排队拿号，它成了瓶颈；它挂了谁都发不了号，成了单点。
- **UUID 是什么**：一串随机生成的 128 位数字，长这样 `550e8400-e29b-41d4-a716-446655440000`。好处是谁都能自己生成、几乎不会撞。
- **为什么不用 UUID，三个原因**：
  - **太长**：128 位 = 16 字节，Snowflake 是 64 位 = 8 字节，差一倍。id 会出现在每一行、每个索引、每个外键里，10 亿行就多占 8 GB，而且传输、比较都慢一倍。
  - **无序**：UUID 是随机的，新生成的号可能比旧的小。比方：往一本按编号排好的账本里插条目。编号递增的话，永远往最后一页加，很快。随机编号就要翻到中间某一页插进去，页满了还要把页拆开（数据库叫页分裂）。数据库的主键索引（B 树）就是这本账本，随机 id 让写入慢几倍，索引也变得松散、占更多空间。
  - **不带信息**：Snowflake 的高位是时间戳，看 id 就知道大概什么时候生成的，按 id 排序 = 按时间排序，查「最近一天的记录」直接按 id 范围扫。UUID 看不出任何东西。
- **什么时候 UUID 也行**：数据量小、不做范围查询、或者用新版 UUID v7（前面带时间戳，有序了，只剩「太长」一个缺点）。
- **英文**：A UUID is 128 random bits. It's twice the size of a 64-bit id, which costs space in every row and index, and because it's random, inserts land in random places in the B-tree index and cause page splits, so writes get slower and the index gets fragmented. A Snowflake-style id is half the size and roughly time-ordered, so inserts append at the end and the id itself tells you when it was created.
- **时钟回拨（clock moving backwards）**是 Snowflake 的经典坑：机器时间往回跳会发重号，要等时间追上来再发。

你的项目里：user_id << 16 | 每人序号。同样是「拆段、各管各的」，但不用时间戳，因为要**可复现**：同样输入每次必须发出同样的 id。user_id 最大 32 位 + 16 位序号 = 48 位，64 位放得下；每人上限 65,536，超了报错进隔离，不静默撞号；id 可以因规则 6 合并而退役，退役后永不复用。

Ids are user_id shifted left 16 bits plus a per-user sequence. It's the Snowflake idea, split the bits and let each part be issued independently, but without a timestamp, because the ids must be reproducible from the input.

5 · 缓存

比方：常用的调料放灶台边，不每次去储藏室拿。

- **cache-aside**（最常用）：先查缓存，没有就查库，查到后写进缓存。写数据时删掉缓存里的旧值。
- **TTL**：缓存多久过期。短 = 新鲜但命中低；长 = 命中高但可能旧。
- **淘汰策略（eviction policy）**：满了删谁。LRU 删最久没用的；LFU 删用得最少的。
- **三个坑**：**穿透（cache penetration）**（查不存在的 key，每次打到库 → 缓存空值）；**雪崩（cache avalanche）**（大量 key 同时过期 → TTL 加随机抖动（jitter））。展开：比方，一栋楼所有人的牛奶都订在早上 8 点整送到，8 点电梯就挤爆。缓存也一样：凌晨 0 点一次性把 100 万个 key 写进缓存、都设 1 小时过期，那 1 点整它们同时失效，100 万个请求同一秒全打到数据库，库被压垮。办法：过期时间别设成固定的 1 小时，设成「1 小时 ± 随机几分钟」，比如 55 到 65 分钟之间随机取。这样 100 万个 key 在 10 分钟内零零散散地过期，数据库每秒只多一点点压力。代码就一行：`ttl = 3600 + random.randint(-300, 300)`。Jitter means adding a small random offset to each TTL so keys that were cached together don't all expire in the same second and stampede the database.；**击穿（hot-key breakdown）**（一个热 key 过期瞬间大量请求 → 加锁只让一个去查）。
- **命中率（hit rate）**：命中次数 ÷ 总次数；80% 命中就把库的压力降到 1/5。

你的项目里：lru_cache 缓存 title_key / sim_key / desc_tokens 的结果（同一个头衔反复出现）；TikTok 那段的在线特征链路用过 20 分钟 TTL + 哈希降级，可以拿来当缓存设计的例子。

I'd put a read cache in front of the serving store with cache-aside and a TTL, add jitter to the TTL so keys don't all expire together, and cache negative results so missing keys don't hammer the database.

6 · 消息队列（Kafka）

比方：快递站的传送带。寄件人放上去就走，分拣员按自己的速度拿，带子上的包裹还能倒回去重看。

- **三个用处：解耦（decoupling）**（生产方和消费方不用同时在线）、**削峰（buffering peaks）**（高峰先排队）、**可重放（replayable）**（消费方挂了从上次的位置接着读）。
- **Topic / 分区 / offset**：一个 topic 切成多个分区；同一个分区内有序，跨分区无序；每个消费者记自己读到的位置（offset）。按 key（比如 user_id）分区，同一个人的消息就在同一分区、按顺序到。
  为什么分区内有序、跨分区无序：比方，银行开了 4 个窗口，每个窗口前排一条队。同一条队里，先来的一定先办（这条队有序）。但 1 号窗口的第 5 个人和 3 号窗口的第 2 个人谁先办完，没人管，看各窗口快慢（跨队无序）。Kafka 的每个分区（partition）就是一个窗口：它是一个只往末尾追加的日志（append-only log），每条消息拿一个递增的位置号 offset，消费者按 offset 从小到大读，所以分区内天然有序。不同分区在不同机器上、由不同消费者并行读，没有一个全局的顺序号把它们串起来；要做全局有序，就只能所有消息排同一条队，也就失去了并行（throughput）。所以做法是：需要有序的东西放进同一个分区。用 user_id 当分区键（partition key），同一个人的消息哈希到同一个分区，他自己的事件就保序；不同人之间本来就不需要顺序。Each partition is an append-only log with increasing offsets, so order holds inside it. Partitions are read in parallel with no global sequence across them; global order would mean one partition and no parallelism. So you choose the partition key to match what must stay ordered: user_id keeps each person's events in order.
- **投递语义（delivery semantics）**：至多一次（at-most-once）（可能丢）、至少一次（at-least-once）（可能重）、恰好一次（贵）。常用做法：至少一次 + 消费端幂等。
- **消费者组（consumer group）**：一组消费者分摊一个 topic 的分区，加消费者就加吞吐，但消费者数不能超过分区数。
- **积压（lag）（consumer lag）**：生产比消费快，offset 落后越来越多，是最重要的监控指标。

你的项目里：每天的新观测可以从 Kafka 进来，按 user_id 分区，这样迟到数据和正常数据在同一个人的分区里；消费端用你的 step 函数，它本来就是幂等可重放的（同样的输入同样的输出）。

New observations would arrive on a Kafka topic partitioned by user_id, so one person's events stay ordered. Delivery is at-least-once and the consumer is idempotent, because step is deterministic, so a redelivered message can't produce a different result.

7 · 批处理 vs 流处理，迟到数据

比方：批处理是每天晚上结一次账；流处理是每笔交易当场记。

- **批**（Spark、MapReduce）：吞吐大、简单、延迟小时到天。**流**（Flink、Kafka Streams）：秒级延迟，但要处理乱序、迟到、状态管理，运维重。
- **事件时间 vs 处理时间（event time vs processing time）**：事情发生的时间 vs 系统看到的时间。两者差 = 迟到。**水位线（watermark）**：系统宣布「X 时刻之前的事件应该都到了」，之后再来的算迟到，要单独处理（丢、或者回头更正）。
- **Lambda 架构**：批 + 流两套，流给实时近似，批晚上修正。**Kappa 架构**：只用流，要重算就从队列头重放。
- **MapReduce 是干什么的**：数据大到一台机器装不下时，用一群机器一起算一个「按某个东西分组统计」的结果。三步：map（映射）→ shuffle（洗牌）→ reduce（归并）。
  例子：算每家公司有多少员工。10 亿行档案分散在 100 台机器上，每台 1000 万行。
  - **map**：每台机器各看各的 1000 万行，每行吐出一对 `(公司名, 1)`，比如 `(Google, 1)`、`(Meta, 1)`。公司名就是 **key**。这一步机器之间完全不用说话，各干各的。
  - **shuffle**：问题来了，Google 的员工散在 100 台机器上，谁来加总？规则：按 key 的哈希，把所有 `(Google, 1)` 都送到同一台机器，比如 7 号机；所有 `(Meta, 1)` 送到 23 号机。这一步要把数据**通过网络在机器之间搬来搬去**，100 台机器互相发数据。
  - **reduce**：7 号机现在手里有所有 Google 的行，把 1 加起来得到 Google 的总数；23 号机算 Meta。各算各的，又不用说话了。

  为什么说 shuffle 最贵：map 和 reduce 都只在本机算（内存、CPU，很快）；shuffle 要把几乎全部数据过一遍网络、还常常要写硬盘排序。网络比内存慢上千倍，所以一个任务的时间大头通常在 shuffle。优化方向就是少 shuffle：能在 map 端先局部合并就先合并（每台机先算出本机 Google 有 5 万人，只发一条 `(Google, 50000)`，叫 combiner / 局部聚合）。
  你的项目：Spark 版的 `repartition by user_id` 就是一次 shuffle，把同一个人散在各个文件里的行送到同一台机器；然后 `mapInArrow` 调你的 coalesce_bucket 就相当于 reduce，每台机器各自处理自己那些人。单机版的「先分桶」做的也是同

一件事。

MapReduce is map, shuffle, reduce. Map runs locally and emits key-value pairs, shuffle moves every pair with the same key to the same machine over the network, and reduce combines each key's group. Shuffle dominates the cost because it pushes almost all the data through the network, so you pre-aggregate on the map side when you can. In my Spark version, repartition by user_id is the shuffle and coalesce_bucket is the reduce.

- **数据倾斜（data skew）**：某个 key 的数据特别多，一台机器干到天亮别的都闲着。办法：热 key 加盐拆开（salting）、两阶段聚合（two-phase aggregation）、或者单独处理大 key。

你的项目里：选批，因为题目只要每天更新，流的复杂度用不上。observed_ts 是事件时间、到达日期是处理时间，迟到 = 两者不一致，你的处理是「只重放那一个人」，相当于迟到数据的回头更正。Spark 的 repartition by user_id 就是 shuffle；人均 29 条、最多 166 条，没有倾斜；真有超级用户就单独处理。

The brief needs daily freshness, so batch is the right tool; streaming would add ordering and state complexity for no gain. observed_ts is event time, arrival is processing time, and late data is handled by replaying just that person, which is the batch version of a watermark correction.

8 · 幂等、恰好一次、确定性重算

比方：电梯按钮按十次和按一次一样。幂等 = 同一个操作做多次（idempotent），结果和做一次一样。

- **为什么重要**：分布式里重试不可避免（网络超时不知道成没成功），只有幂等才能放心重试。
- **怎么做到**：操作带唯一 id，做过就跳过（去重表（dedup table））；或者操作本身是「设置成 X」而不是「加 1」。
- **恰好一次（exactly-once）**通常 = 至少一次 + 幂等，或者事务性写入（输出和 offset 一起提交）。
- **确定性（deterministic）**：同样输入同样输出，没有随机、没有依赖时间或机器。有了确定性，重跑就是最简单的修复手段。

你的项目里：id 从 user_id 和首次出现顺序算出来，没有全局计数器 → 确定性；测试「跑两次结果一样」「全量 = 半量 + 增量」就是幂等和确定性的证明；输出文件先写临时再改名（原子写），跑一半挂了不会留半个文件；迟到数据直接重放，因为重放是安全的。

The pipeline is deterministic: same input, same ids, same output, with no global counter and no randomness. That's what makes retries and replays safe, and it's covered by tests: run twice equals once, and full equals half plus incremental.

9 · 存储：对象存储、列式文件、数据湖表格式

比方：S3 是无限大的仓库，parquet 是按列装箱的货物，Iceberg 是仓库的账本（哪些箱子算当前这一版）。

- **对象存储（S3）（object storage）**：便宜、无限、自带多副本；但没有「追加」和「改一行」，只能整文件写。延迟高（几十毫秒），适合批不适合点查。
- **列式（parquet）（columnar）**：同一列存在一起。好处：只读需要的列（你 conform 只读 10 列）；同列压缩率高；带统计信息（每块的最大最小值）能跳过整块（**谓词下推（predicate pushdown）**）。坏处：改一行要重写文件。
- **行式（row-oriented）**（数据库表）：一行存一起，点查快、改一行快。
- **表格式（Iceberg / Delta / Hudi）（table format）**：在一堆 parquet 文件上加一层元数据，提供：**ACID 提交（ACID commit）**（一次写入要么全可见要么全不可见）、**快照 / 时间旅行（snapshot / time travel）**（回到任何一版）、**MERGE**（按键更新）、**分区裁剪（partition pruning）**（按分区只读需要的文件）、**schema 演进（schema evolution）**（加列不重写）、**小文件合并（compaction）**。
- **Iceberg 的「事务提交」具体是怎么做到的**：
  - 比方：仓库里放着一堆箱子（parquet 文件），墙上有一块公告板，贴着一张清单：「当前这一版 = 这 500 个箱子」。读的人只认清单，清单上没有的箱子当不存在。
  - 写新数据的流程：① 先把新箱子搬进仓库（写新的 parquet 文件，旧文件一个都不改）；② 写一张新清单，列出新版本包含哪些箱子（旧的 480 个 + 新的 30 个，去掉被替换的 20 个）；③ 最后一步，把公告板上的清单换成新的。这一换是一个原子动作，要么换了要么没换，没有中间状态。
  - 所以：写到一半挂了，新箱子搬了一半但清单没换，读的人还是看旧清单，什么都没变，只是仓库里多了几个没人认领的箱子，事后清理。这就是「要么全可见要么全不可见」。
  - 读的人在你换清单的那一刻之前拿到的是旧清单，之后拿到的是新清单，永远不会拿到一张写了一半的清单。读写互不阻塞。
  - 旧清单不扔，所以可以「回到上周三那一版」（时间旅行），也可以一键回退。
  - 两个人同时写：换清单时要先检查「公告板上现在还是我开始时看到的那张吗」。是，就换；不是（别人先换了），就拿新清单重做一遍再换。这叫乐观并

发（optimistic concurrency），和 git push 被拒绝要先 pull 一样。
    - 术语对照：清单 = 元数据文件（metadata.json → snapshot → manifest），公告板 = catalog（Hive Metastore / Glue / Nessie），换清单 = 对 catalog 做一次 compare-and-swap。

  Iceberg never modifies a data file. A write adds new parquet files, writes a new metadata snapshot listing exactly which files make up the new version, and then atomically swaps the table's pointer in the catalog. Readers only follow the pointer, so they see either the old version or the new one, never a half-written state. Old snapshots stay, which gives time travel and rollback, and concurrent writers use compare-and-swap on the pointer.

- **小文件问题（small-files problem）**：几百万个 1 MB 文件比几千个 1 GB 文件慢得多（打开文件有固定开销）。目标每个文件 128 MB–1 GB。

你的项目里：输出 parquet、每桶一个文件（64 个），大小合适；状态现在是 parquet 每天整份重写，生产应放 Iceberg 按 user_id 分区、MERGE 只更新动过的人、一次运行一次提交、旧版本留着回退；history 只追加。桶太多 = 小文件问题，所以桶数不是越多越好。

Output goes to parquet on object storage, one file per bucket. For state I'd use Iceberg partitioned by user_id: merge only touched users, one atomic commit per run, and snapshots so a bad run rolls back. Parquet's column pruning and min-max statistics are why reading only ten columns is cheap.

10 · 实体消歧 / 去重系统（record linkage）

比方：两本通讯录合并，「张伟 13800」和「Zhang Wei 138-00」是不是同一个人。

- **三步**：**Blocking（blocking / candidate generation）**（先用便宜的键把候选缩小，比如同雇主）→ **成对比较（pairwise comparison）**（打分：相似度 + 规则或模型）→ **聚类 / 传递闭包（clustering / transitive closure）**（A=B、B=C → A=C，合成一簇）。
- **Blocking 为什么必须**：N 条两两比是 N^2；1000 亿行不可能。按 user_id 和雇主分块后，每人只有几十条。
- **规则 vs 模型**：规则可解释、可复现、快；模型（逻辑回归 / 梯度提升 / embedding）召回高但要标注、要可复现地部署。常见做法：规则做主、模型只用在规则拿不准的那层。
- **评估**：成对指标（pairwise metrics）（配对的精确率 / 召回）vs 簇指标（B-cubed）（cluster metrics, B-cubed）；没有标准答案时抽样 + 判官 + 人工 + 无标注健康度。
- **传递性的坑（chaining problem）**：A 像 B、B 像 C，但 A 和 C 一点不像，链式合并会把不相关的合成一团。要么限制簇大小，要么要求和簇里多数相似。

你的项目里：规则 2 就是 blocking（同雇主 + 日期兼容才算候选）；规则 4–5 是成对比较；规则 6 是把已发出的 id 做传递合并，但只在被动过的职位上做、且要同开始日期，避免链式乱合。学习打分器试过、没上线，下一步只用在弱点层。

It's a standard record-linkage shape: blocking by employer and date to keep candidates small, pairwise scoring with rules, then a constrained merge of duplicate ids. Rules first because they're reproducible and explainable; a learned scorer only where the rules are unsure.

11 · 数据质量监控与可观测性

比方：工厂流水线上的质检员，不等客户投诉才发现次品。

- **三类指标（volume · quality · freshness）**：**量**（行数、人数、文件数，和昨天比差多少）、**质**（空值率、重复率、日期倒序数、覆盖率）、**时效**（数据几点到、跑了多久）。
- **告警规则**：和历史同期比，偏离 3 个标准差或者某个百分比就报警；绝不能为 0 的指标（主键重复、倒序日期）直接硬断言。
- **数据契约（data contract）**：上游保证 schema 和含义不变，变了要通知；你的 conform 「缺列直接报错」就是契约的硬执行。
- **金丝雀 / 回归测试（canary / regression test）**：固定一小份输入，每次跑完对比输出是否变化。
- **隔离而不是整体失败（quarantine, partial failure）**：坏的一小部分进隔离区，其余照常出，事后看隔离区。
- **血缘（lineage）**：这列数据从哪来、经过哪些步骤，出了问题能追。

你的项目里：run.json 里的指标（输入行、输出行、丢弃行按规则、修复格数、合并数、重放人数、隔离人数）就是质量看板；覆盖率 99.94%、倒序日期 0、可疑拆分 3.7 万是无标注健康度；quarantine 目录是隔离区；51 个测试里的金丝雀。被问「上线后怎么知道结果还好」：每天看这几个数的走势，突变就查。

Every run emits metrics: rows in and out, drops per rule, repairs, merges, replays, quarantined users. I'd chart those daily and alert on deviations, hard-fail on anything that must be zero like duplicate keys or inverted dates, and keep the quarantine folder so one bad user never blocks the run.

12 · 服务层：把结果给下游用

比方：仓库里的货要上货架，顾客才拿得到。批处理的结果要有人能查。

- **两种读法**：**分析查询（analytical query）**（整表扫描、聚合）→ 直接读 parquet / 数据仓库（BigQuery、Snowflake、Trino）；**点查（point lookup）**（给我这个人的履历）→ 要 KV 存储（DynamoDB、Cassandra、Redis），按 user_id 做键，毫秒级。
- **批到线上的搬运**：每天跑完把结果批量写进 KV（bulk load），或者只写动过的人。
- **API 设计**：GET /users/{id}/positions?asof=2026-10-01；分页；版本号；限流（rate limiting）。
- **Cassandra / DynamoDB 一句话**：按分区键哈希分片、多副本、最终一致、写快读快，不擅长复杂查询和 join。
- **读写分离（read/write separation）**：批处理写一份，线上读另一份，互不影响。

你的项目里：下游要查「某人截至某天的履历」→ 按 (user_id, asof_ts) 做键放 KV；每天只有动过的人要更新，正好和增量输出对上；restated 名单告诉下游哪些人的历史行要整体替换。

For point lookups I'd load the daily output into a key-value store keyed by user and as-of date, updating only the users touched that day, and the restated list tells the loader which users need their history replaced.

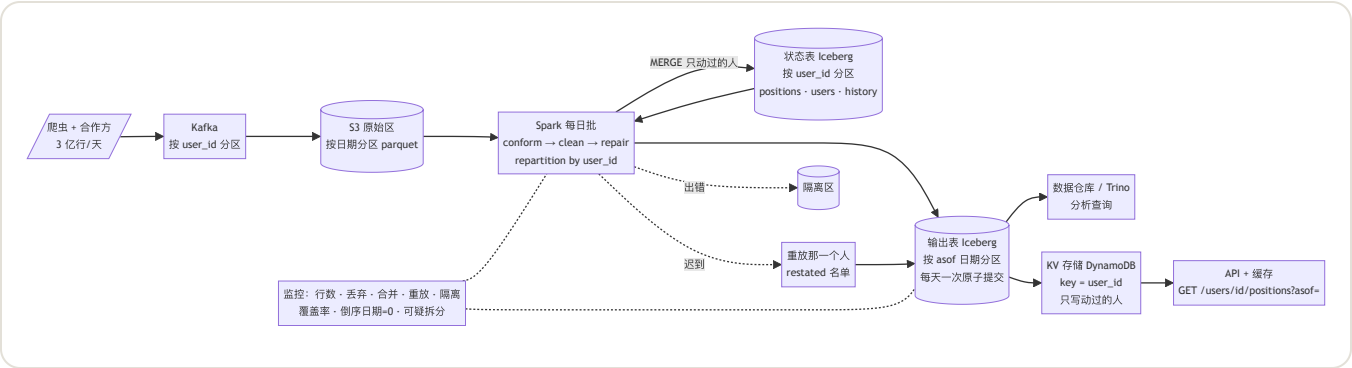
押题最终答案（三道，每道一张图 + 最优设计）

补充手册第 2 章有思路，这里给的是「最后要落到的那张图」。答不出来就画图、按框念。

题 A · 每天几亿条档案进来，维护去重后的「每人职业经历」（= 你的 take-home 上生产）

先问：每天多少条（设 3 亿）、新鲜度（天级够）、下游谁读（分析 + 点查）、要不要保留历史版本（要，as-of）。

估算：3 亿行/天 × 500 B = 150 GB/天；存量 1000 亿行 ≈ 50 TB；每天只有约 1% 的人有新数据；按 3.5 万行/核秒，每天增量 3 亿行 ≈ 2.4 核时，几台机器半小时。

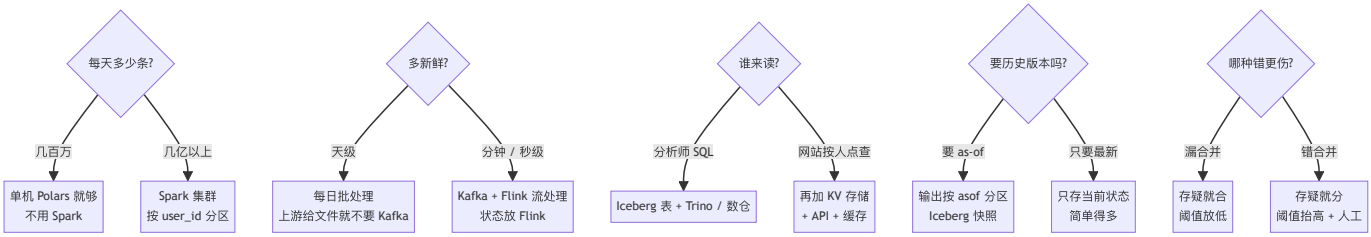


- **核心**：和作业一样的 coalesce_bucket，Spark mapInArrow 调同一个函数；按 user_id 分区所以状态 join 不用跨机器。
- **「输出表按 asof 日期分区」是什么意思**：asof_ts 是你输出里的那一列，「截至哪一天」的履历。按它分区 = 把输出表按日期分成一个个文件夹：`asof=2026-10-28/`、`asof=2026-10-29/`、`asof=2026-10-30/`，每天的结果放自己的文件夹里。比方：档案室按日期分抽屉，每天一个抽屉。好处：① 每天跑完只往今天的抽屉里放东西，不碰旧抽屉；② 查「某人 10 月 29 日的履历」只开那一个抽屉（分区裁剪，partition pruning），不用翻全部；③ 跑坏了只重做今天的抽屉。和「按 user_id 分区」的区别：状态表要按人找，所以按 user_id 分；输出表下游是按日期查，所以按日期分。Partitioning the output by asof date means one folder per day; each run only writes today's folder, a query for one day reads only that folder, and a bad day can be rewritten alone. State is partitioned by user because it's looked up by user; output is partitioned by date because it's queried by date.
- **状态**：Iceberg 按 user_id 分区、只 MERGE 动过的人、一次运行一次提交、旧快照留着回退；history 只追加、只给迟到的人读。
- **迟到**：事件时间 observed_ts；比状态旧的只重放那一个人，restated 告诉 KV 加载器整体替换。
- **隔离**：一个人出错回滚进隔离区，不阻塞当天运行。

- **服务**：分析走仓库；点查走 KV（key = user_id，value = 当前履历 + as-of 索引）+ 读缓存。
- **扩容**：人独立 → 加 executor 线性；分区数调到每分区几十万行；超级用户单独处理。
- **取舍**：天级批而不是流（新鲜度要求天级，流的乱序和状态管理不值）；规则而不是 LLM 逐对（可复现、可解释、1000 亿行成本）；merge when in doubt。
- **隐私（right to be forgotten）**：删除名单在输出时过滤。

Same core as the take-home, run under Spark partitioned by user_id, with state in an Iceberg table partitioned by user_id so each day merges only touched users and commits atomically. Late data replays one person; failures are quarantined. Output goes to a warehouse for analysis and a key-value store for point lookups, and every run emits the quality metrics I alert on.

先问的思维导图：问五个问题，每个答案决定一个组件。 左边是问题，箭头上是他可能的回答，右边是这个回答把设计引向哪里。



题 A：逐步思考和取舍（每一步：要解决什么 → 可选方案 → 选哪个 → 为什么 → 代价）

第 1 步 · 先问清楚要求（clarify requirements）

要解决什么：同一道题，每天 1 万条和每天 10 亿条、要秒级更新和要天级更新，设计完全不同。不问就画，十有八九画错方向。

可选方案：

- 问：每天进来多少条？（设 3 亿）
- 问：结果要多新鲜？秒级、小时级还是天级？（设天级，和 take-home 一样）
- 问：谁读结果？分析师跑 SQL，还是网站按人点查？（设两种都有）
- 问：要不要保留历史版本，能查「截至某天」的样子？（要，as-of）
- 问：错一点能不能接受？漏合并和错合并哪个更伤？

选哪个：问完把答案写在白板最上面，后面每个决定都回头指着它说「因为天级就够，所以……」。

为什么：面试官最看重你会不会先界定问题。每个答案都会直接决定后面一个组件的选择。

代价 / 取舍：花 3–5 分钟，看起来慢，但能避免后面整段推倒重来。

Before drawing anything, I'd confirm the daily volume, how fresh the output must be, who reads it, and whether we need history, because each answer changes a component.

第 2 步 · 估算（back-of-envelope）

要解决什么：知道数据有多大，才知道一台机器够不够、要几台、存储花多少钱。

可选方案：

- 只算数量级，不求精确

选哪个：3 亿行/天 × 500 字节 ≈ 150 GB/天；存量 1000 亿行 ≈ 50 TB；每天约 1% 的人有新数据；你的代码 3.5 万行/核秒 → 3 亿行 ≈ 8,600 核秒 ≈ 2.4 核时。

为什么：结论很关键：每天的增量很小，几台机器半小时跑完；真正大的是存量和偶尔的全量重算（800 核时）。所以设计重点是「增量只碰动过的人」，而不是「每天全量重跑」。

代价 / 取舍：估算有误差，所以说「数量级」，并说明假设（每行 500 字节、1% 的人有更新）。

Three hundred million rows a day at 500 bytes is about 150 gigabytes; at 35 thousand rows per core-second that's under three core-hours, so the daily job is small. The design should touch only users with new data rather than recompute everything.

第 3 步 · 数据怎么进来（ingest）

要解决什么：爬虫和合作方源源不断地送数据，要有个地方先接住，不能直接塞进处理程序（处理程序挂了数据就丢了）。

可选方案：

- 直接写数据库：简单，但数据库扛不住每天几亿行的写入，而且和处理逻辑绑死。
- 写进 Kafka（消息队列）：生产者和消费者解耦（decoupling），消费者挂了数据还在，可以重放（replay）。
- 直接落 S3 文件：最便宜，按天一个文件夹，但不方便实时消费。

选哪个：Kafka 按 user_id 分区接住 → 每小时把 Kafka 里的数据落成 S3 上按日期分区的 parquet（原始区，raw zone）。

为什么：Kafka 负责「接住、不丢、能重放」；S3 原始区负责「便宜地长期保存原样数据」，以后规则改了可以从原始数据重算。按 user_id 分区让同一个人的事件在同一个分区里保序。

代价 / 取舍：多维护一个 Kafka 集群。如果新鲜度只要天级、上游本来就每天给文件，可以不要 Kafka，直接收 S3 文件，更简单。面试时把这个取舍说出来。

I'd land events on Kafka partitioned by user_id so nothing is lost and one person's events stay in order, then persist them hourly as date-partitioned parquet in a raw zone on S3. If upstream already delivers daily files, I'd skip Kafka and read S3 directly.

第 4 步 · 选计算引擎：批还是流

要解决什么：每天要把新数据和已有状态合起来算一遍，用什么跑?

可选方案：

- Spark 每日批处理（batch)：攒一天的数据一次算完。
- Flink 流处理（streaming)：来一条算一条，秒级。
- 单机 Polars：你作业那样，够快但只有一台机器。

选哪个：Spark 每日批。

为什么：要求是天级新鲜度，流处理的好处（秒级）用不上，却要付出它的代价：一直开着、状态管理、乱序处理、运维（operations）重。Spark 成熟、按天跑正好、和 Iceberg 配合好，而且 mapInArrow 能直接调你写好的 coalesce_bucket，规则只有一份。

代价 / 取舍：新数据最晚要等一天才可见。如果以后要求变成分钟级，再把核心函数搬进 Flink（函数本身不用改，因为它就是「给一个人的状态和新快照，返回新状态」)。

Daily freshness makes batch the right call; streaming would add state management and operations for no benefit. Spark runs the same coalesce_bucket through mapInArrow, so there's one copy of the logic, and it could move to Flink later without changes.

第 5 步 · 核心处理怎么并行（partition by user_id）

要解决什么：1000 亿行一台机器跑不动，要分给很多机器。怎么分才不会算错?

可选方案：

- 随机分：均匀，但同一个人的行散在多台机器，没法做每人的状态机。
- 按 user_id 分：同一个人的数据一定在同一台机器。
- 按公司分：同一公司在一起，但一个人换过多家公司会被拆开。

选哪个：按 user_id 分区（repartition by user_id，这一步就是一次 shuffle）。

为什么：你的算法是「每人一个状态机」，人和人之间不共享任何东西，所以只要同一个人在一台机器上就能独立算，加机器就线性变快。

代价 / 取舍：一次 shuffle 要走网络，是最贵的一步；所以状态表也按 user_id 分区，让 join 时同一个人的状态和新数据本来就在一起，少搬数据。超级用户（几千条的人）可能造成数据倾斜，单独处理。

Partitioning by user_id puts all of one person's rows on one worker, and since nothing is shared between people, it scales linearly. The shuffle is the expensive step, so the state table is partitioned the same way.

第 6 步 · 状态存哪里（state store）

要解决什么：每个人「目前有哪些职位、下一个序号、上次处理到哪天」要存下来，明天接着用。

可选方案：

- 现在作业里的做法：每天把 parquet 整份重写到新文件夹。简单，但每天重写全部，哪怕只有 1% 的人变了。
- 放 KV 数据库（DynamoDB）：按人读写快，但批处理一次读几亿人很贵。
- 放 Iceberg 表，按 user_id 分区。

选哪个：Iceberg 表，按 user_id 分区，每天用 MERGE 只更新动过的人；history 单独一张只追加（append-only）的表。

为什么：① MERGE 只改动过的人，不用整份重写；② 按 user_id 分区，当天只读有新数据的人所在分区（分区裁剪，partition pruning）；③ 一次运行一次原子提交，跑一半挂了不会留下半截状态；④ 旧快照留着，跑坏了一键回退到昨天。

代价 / 取舍：多了一层表格式要维护（定期合并小文件 compaction、清理旧快照）。

State lives in an Iceberg table partitioned by user_id: each day merges only touched users, reads only their partitions, commits atomically, and keeps snapshots so a bad run can be rolled back.

第 7 步 · 迟到数据和出错怎么处理

要解决什么：有的快照晚几天才到；有的人数据奇怪会让程序报错。不能让一个人拖垮整批。

可选方案：

- 迟到：直接丢弃（简单但输出错）
- 迟到：隔离等人工（安全但积压）
- 迟到：只重放那一个人（你作业的做法）
- 出错：整批失败重跑 vs 只隔离出错的人

选哪个：迟到只重放那个人、写进 restated 名单；出错的人回滚进隔离区（quarantine），其余照常提交。

为什么：人和人独立，所以重放一个人、隔离一个人都不影响别人。重放结果和全量跑一致，有测试证明。

代价 / 取舍：要多存 history；restated 的人下游要整体替换他的历史行。隔离区要有人定期看。

Late snapshots replay just that user and are listed as restated; a user who fails is rolled back and quarantined while everyone else commits.

第 8 步 · 结果怎么给下游用（serving）

要解决什么：分析师要跑 SQL 统计；网站要「给我这个人截至某天的履历」毫秒级返回。一个存储满足不了两种读法。

可选方案：

- 只放数据仓库：SQL 方便，但点查要几秒。
- 只放 KV：点查快，但没法做统计。
- 两份都放。

选哪个：输出 Iceberg 表（按 asof 日期分区）给 Trino / 数仓跑分析；同时每天把动过的人批量写进 DynamoDB（key = user_id），前面加 API 和缓存给网站点查。

为什么：读法不同就用不同的存储，这叫读写分离 / 多存储（polyglot persistence）。每天只写动过的人，KV 的写入量很小。

代价 / 取舍：同一份数据两份拷贝，要保证一致：以 Iceberg 为准，KV 是它的派生，出问题从 Iceberg 重新灌。

Analysts query the Iceberg output through a warehouse; point lookups go to a key–value store loaded daily with only the touched users, behind an API and cache. The table is the source of truth and the store can be rebuilt from it.

第 9 步 · 监控（monitoring）

要解决什么：没有标准答案，上线后怎么知道结果还对、哪天抓取器坏了？

可选方案：

- 只看程序有没有报错（不够，结果错了程序不会报错）
- 每次运行输出质量指标并告警

选哪个：每次运行出指标：输入行数、丢弃行数（按规则）、修复格数、合并数、重放人数、隔离人数、覆盖率、倒序日期数、可疑拆分数、新 id 比例；和历史比，突变就告警；必须为 0 的（主键重复、倒序日期）直接让任务失败。再加定期抽样 + 判官。

为什么：数据错误不会抛异常，只能靠指标发现。抓取器改版通常最先表现为「新 id 比例突然涨了」。

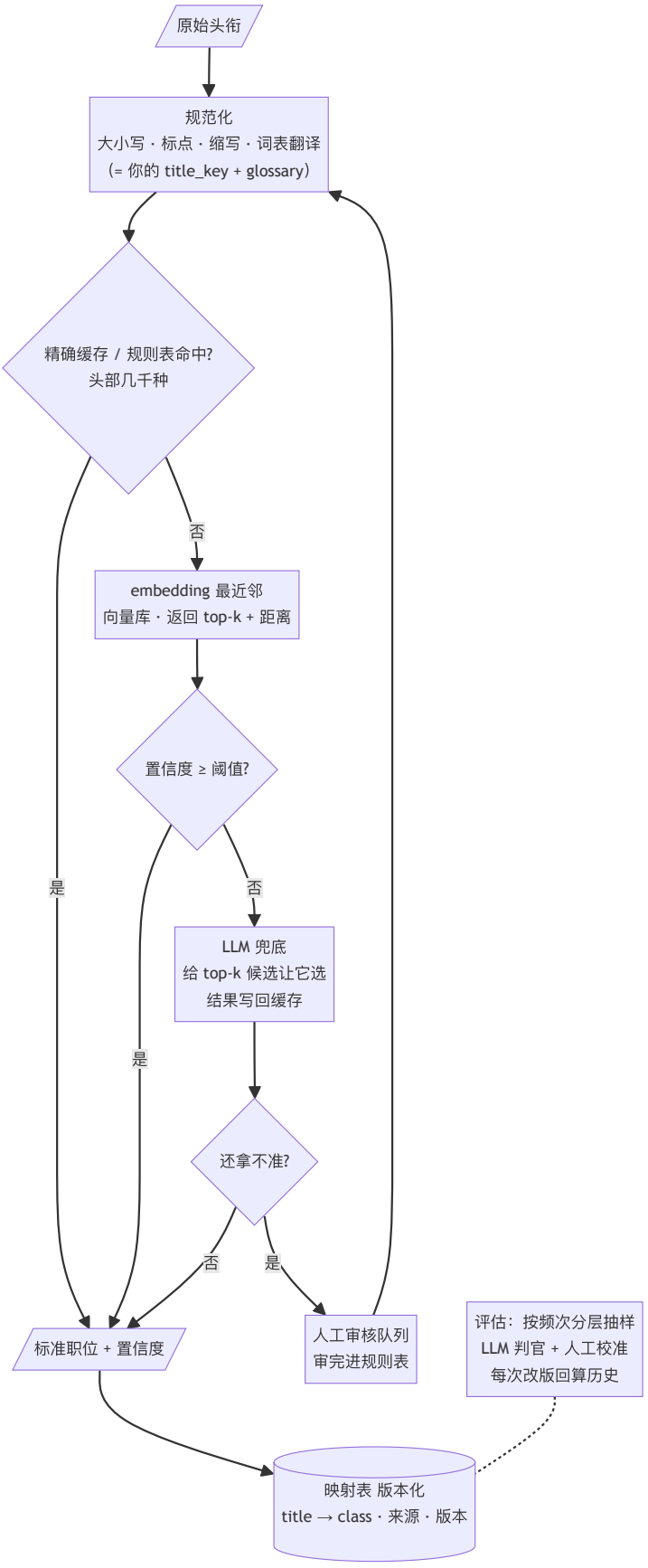
代价 / 取舍：告警阈值要调，太松漏报、太紧吵人。

Every run emits counts per rule, coverage, inverted dates, suspicious splits and the new-id rate; I alert on deviations from history and hard-fail on anything that must be zero, plus periodic sampled judging.

题 B · 职位名 / 技能映射服务（1000 万种原始职位名 → 标准分类）

先问：分类表多大（几千类）、延迟要求（线上毫秒 vs 离线批）、准确率目标、要不要多语言、分类表会不会改版。

估算：去重后约 1000 万种头衔，但头部几千种覆盖 80% 流量；线上 QPS 设 1 万；每个向量 384 维 × 4 B ≈ 1.5 KB，1000 万 × 1.5 KB = 15 GB，一台机器放得下。

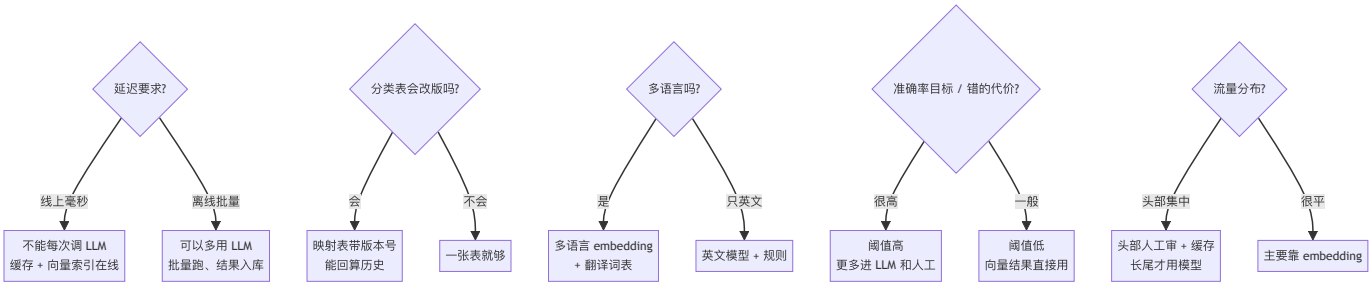


- **三层漏斗**：规则/缓存吃掉大头（毫秒、确定）；embedding 吃长尾（几毫秒）；LLM 只处理低置信（贵、慢、结果缓存后不再调）。

- **为什么不全用 LLM**：1000 万次调用成本，答案不稳定，线上延迟；LLM 的价值是兜底 + 当判官。
- **版本化**：分类表改版要能回算历史，不然时间序列断；映射表记来源（规则 / 向量 / LLM / 人工）。
- **评估**：按频次分层抽样（头部错一个影响大）；LLM 判官用人工样本校准；监控「进 LLM 的比例」「进人工的比例」，涨了说明有新词。
- **多语言**：多语言 embedding 模型；词表先翻译。
- **取舍**：准确 vs 延迟 vs 成本，用置信度阈值调；阈值高 → 更多进 LLM/人工。

A three-tier funnel: normalise, then rules and a cache for the head of the distribution, embedding nearest-neighbour for the long tail, and an LLM only for low-confidence cases, with results cached and the lowest sent to human review. The mapping table is versioned so history can be recomputed, and I evaluate on frequency-stratified samples with an LLM judge calibrated against human labels.

先问的思维导图：问五个问题，答案决定「花多少钱在准确率上」。 左边是问题，箭头上是他可能的回答，右边是这个回答把设计引向哪里。



题 B：逐步思考和取舍（每一步：要解决什么 → 可选方案 → 选哪个 → 为什么 → 代价）

第 1 步 · 先问清楚

要解决什么：职位名有一千万种写法，要映射到几千个标准类。线上实时用还是离线批量用、准确率要多少，决定了能不能用贵的方法。

可选方案：

- 问：标准分类表多大？会不会改版？
- 问：延迟要求？线上搜索框要毫秒级，离线报表可以慢
- 问：多语言吗？
- 问：准确率目标？错了的代价？

选哪个：设：几千个标准类、会改版；线上要毫秒级、离线也要；多语言；头部准确率要很高。

为什么：延迟和成本决定能不能每次都调 LLM（不能）。改版决定映射表要版本化。

代价 / 取舍：—

I'd confirm the taxonomy size and how often it changes, the latency target, languages, and the accuracy bar, since those decide whether we can afford an LLM per lookup.

第 2 步 · 估算

要解决什么：看数据分布，决定在哪里花钱。

可选方案：

- 统计每个头衔出现多少次

选哪个：一千万种头衔，但头部几千种覆盖约 80% 的流量（长尾分布，long tail）；线上 1 万 QPS；向量 384 维 × 4 字节 ≈ 1.5 KB，一千万个 ≈ 15 GB，一台机器内存放得下。

为什么：既然大头集中在少数头衔，就把它们做到又快又准；长尾才用更贵的方法。

代价 / 取舍：—

The distribution is long-tailed: a few thousand titles cover most traffic, so the head gets cheap exact handling and only the tail needs expensive methods.

第 3 步 · 规范化（normalisation）

要解决什么：「Sr. Software Engineer」「senior software engineer」「高级软件工程师」其实是一回事，先把写法统一，后面才好比。

可选方案：

- 什么都不做直接比（命中率低）
- 规范化：大小写、标点、缩写展开、词表翻译（你的 title_key + glossary）

选哪个：先规范化，生成一个 key。

为什么：规范化便宜、确定，一下子把很多写法合成同一个 key，后面缓存命中率大幅提高。

代价 / 取舍：规则要维护；翻译过度会把不相关的合在一起（你作业里 chief 被误翻那次）。

Normalise first, case, punctuation, abbreviations and a translation glossary, so many spellings collapse to one key before anything expensive runs.

第 4 步 · 头部：缓存 / 规则表

要解决什么：最常见的几千个头衔要毫秒级、百分百确定。

可选方案：

- 每次都现算
- 人工审过的规则表 + 缓存（Redis）

选哪个：规范化后的 key 先查规则表 / 缓存，命中直接返回。

为什么：占了大部分流量；人工审过一次就永远对；内存查找微秒级。

代价 / 取舍：规则表要有人维护；分类表改版时要批量更新。

The head is served from a reviewed mapping table in a cache: microseconds and exact.

第 5 步 · 长尾：embedding 最近邻

要解决什么：没见过的写法，规则对不上，需要「按意思找最像的标准类」。

可选方案：

- 编辑距离 / token 相似度（便宜，但不懂意思：Developer 和 Engineer 字面不像）
- embedding 最近邻（懂意思，几毫秒）
- 直接问 LLM（最懂，但贵、慢、不稳定）

选哪个：用多语言 embedding 模型把头衔变成向量，在标准类的向量里找最近的 top-k，并给一个距离当置信度。

为什么：embedding 懂语义、支持多语言、几毫秒、成本低，能吃掉大部分长尾。

代价 / 取舍：要部署模型和向量索引；距离阈值要用标注数据调。

For the long tail, a multilingual embedding finds the nearest standard titles in milliseconds, with distance as a confidence score.

第 6 步 · 低置信：LLM 兜底 + 人工

要解决什么：embedding 拿不准的（最近的两个类差不多近），需要更聪明的判断。

可选方案：

- 全交人工（准但慢、贵）
- 全交 LLM（贵、不稳定）
- 只把低置信的交 LLM，LLM 也拿不准的交人工

选哪个：低于阈值的交给 LLM：给它 top-k 候选让它选（不让它自由发挥）；结果写回缓存，同一个头衔不会问第二次。LLM 也拿不准的进人工队列，审完写进规则表。

为什么：把贵的资源只用在难的少数；LLM 结果缓存后成本是一次性的；人工审核的结果回流，系统越用越准。

代价 / 取舍：LLM 答案要可复现：固定模型版本、温度 0、记录版本号。

Only low-confidence cases go to an LLM, constrained to the top candidates and cached; the hardest go to human review, which feeds back into the rule table.

第 7 步 · 版本化和评估

要解决什么：分类表改版后，历史数据要能按新版本重算，否则时间序列断开；还要知道准确率有没有下降。

可选方案：

- 映射表只存最新（简单，但历史对不上）
- 映射表带版本号和来源（规则 / 向量 / LLM / 人工）

选哪个：映射表存 (头衔, 标准类, 来源, 分类表版本, 模型版本)；按频次分层抽样评估，LLM 判官用人工样本校准；监控「进 LLM 的比例」「进人工的比例」。

为什么：头部错一个影响大，所以分层抽样；比例突然上涨说明出现了新词或者模型退化。

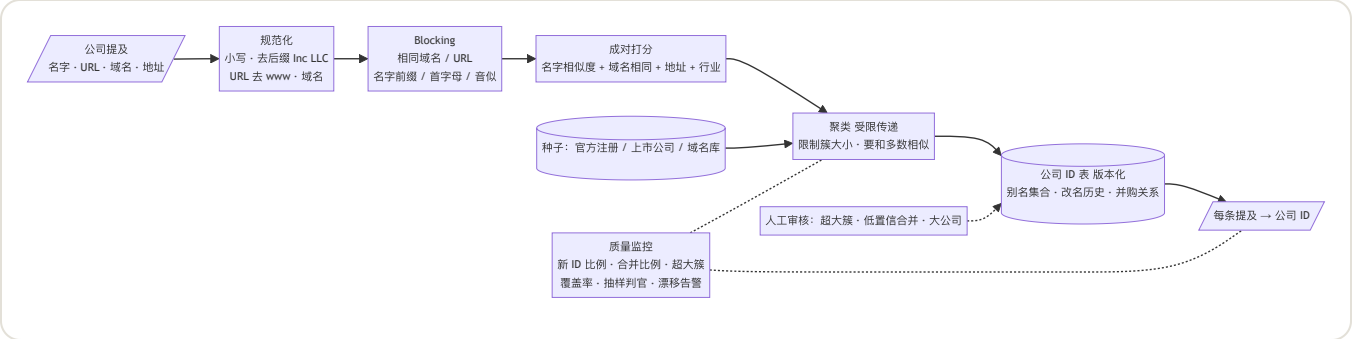
代价 / 取舍：多存几列；评估要持续投入人工。

The mapping is versioned with its source and taxonomy version so history can be recomputed, and I evaluate on frequency-stratified samples and watch the share sent to the LLM and to humans.

题 C · 公司实体消歧 + 数据质量监控

先问：公司实体来源（档案里的公司名、URL、官网、注册信息）、要合并到什么粒度（法人 vs 品牌 vs 子公司）、改名/并购怎么算、下游要什么（公司级统计）。

估算：几亿条公司提及，去重后几千万个名字，真实公司几百万；两两比不可能 → 必须 blocking。

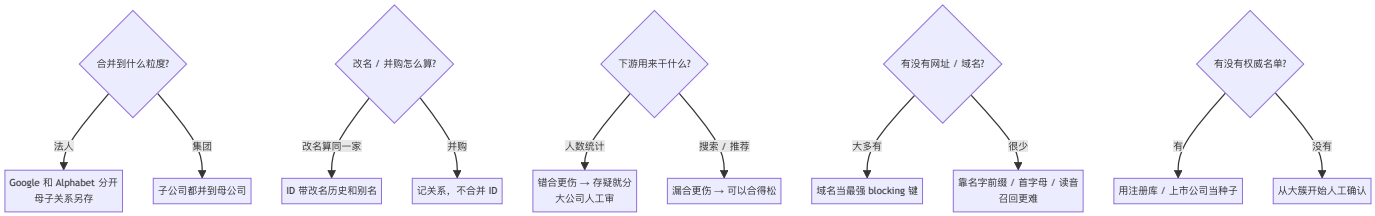


- **三步**：blocking（域名最强、名字前缀 / 首字母 / 音似兜底）→ 成对打分（名字 + 域名 + 地址 + 行业）→ 受限聚类（限制簇大小，防 A 像 B、B 像 C 的链式乱合）。
- **别名集合 + 改名历史**：一个公司记所有叫过的名字和域名；并购是关系不是合并（Jeppesen/Boeing 那种）。
- **种子数据**：官方注册库、上市公司、域名库当锚点，先把大公司钉死。
- **质量监控**：每天看新 ID 比例、合并比例、最大簇大小、覆盖率；分层抽样 + 判官；突变告警（抓取器变了会先体现在这里）。
- **人工**：只审三类：超大簇、低置信合并、头部大公司。
- **版本化**：ID 永不复用、可退役；历史可回算。
- **取舍**：合并错（两家公司算一家，统计错）比漏合并（一家算两家）更伤 → 这里和作业相反，存疑就分开，靠人工和种子来合。

Standard record linkage: normalise names and domains, block on domain and name prefix, score pairs on name similarity plus domain, address and industry, then cluster with a cap on cluster size to avoid chaining. Each company keeps an alias set and rename history; mergers are relationships, not merges. Official registries seed the big companies. For quality I track new-id and merge rates, max cluster size and coverage daily, sample with judges, and route oversized or low-confidence clusters to human

review. Here a wrong merge hurts more than a missed one, so unlike the take-home I split when in doubt.

先问的思维导图：问五个问题，最关键的是第 3 个：错合和漏合哪个更伤。 左边是问题，箭头上是他可能的回答，右边是这个回答把设计引向哪里。



题 C：逐步思考和取舍（每一步：要解决什么 → 可选方案 → 选哪个 → 为什么 → 代价）

第 1 步 · 先问清楚

要解决什么：「公司」的定义不同，答案完全不同：Google 和 Alphabet 算不算一家？YouTube 呢?

可选方案：

- 问：合并到什么粒度？法人、品牌、还是集团?
- 问：改名、并购怎么算?
- 问：下游用来干什么？公司人数统计最怕什么错?

选哪个：设：按法人合并；改名算同一家；并购记成关系（母子公司）不合并；下游做公司人数统计。

为什么：下游是统计人数，两家公司被错合成一家会让人数直接翻倍，比「一家被拆成两家」更伤。这决定了后面阈值要偏保守。

代价 / 取舍：—

I'd pin down the entity level, legal entity versus brand versus parent, and how renames and acquisitions count, because a wrong merge doubles a company's headcount.

第 2 步 · 估算 + 为什么要 blocking

要解决什么：几千万个不同的公司名，两两比较是多少次?

可选方案：

- 全部两两比
- 先分组再比

选哪个：几千万个名字两两比 ≈ 10^{14} 到 10^{15} 次，算不完。必须先用 blocking 把候选缩小到每组几十个。

为什么：blocking 是这类问题能不能做出来的关键。

代价 / 取舍：blocking 太严会漏掉本该合并的（召回下降），太松又算不完。所以用几种键各分一遍，取并集。

Comparing every pair is about ten to the fifteen comparisons, so blocking on cheap keys is what makes it feasible; I'd use several keys and union the candidates to protect recall.

第 3 步 · 规范化 + blocking 键

要解决什么：同一家公司有很多写法：Google LLC / google inc / google.com。

可选方案：

- 只比名字
- 名字 + 域名 + URL + 地址

选哪个：规范化：小写、去标点、去 Inc/LLC/Corp、URL 去 www 和 https、取域名。blocking 键：① 相同域名（最强）② 名字前两个词相同 ③ 首字母缩写相同（RL ↔ Revelio Labs）④ 读音相似。

为什么：域名几乎是公司的身份证，最可靠；名字类的键兜住没有网址的情况。

代价 / 取舍：多个键会产生更多候选，计算量上去一些。

Normalise names and URLs, then block on shared domain first, plus leading name tokens, initials and phonetic keys for records without a URL.

第 4 步 · 成对打分（pairwise scoring）

要解决什么：每个候选对，算「是同一家」的可能性。

可选方案：

- 手写规则打分（像你的 take-home）
- 训练一个分类器（逻辑回归 / 梯度提升树）

选哪个：先规则：名字相似度 + 同域名 + 地址 + 行业 + 员工共现（两个名字下面有同一批人）。有了标注后再训一个分类器输出概率。

为什么：规则可解释、马上能上线；分类器在有标注后能把信号组合得更好。

代价 / 取舍：分类器需要标注、要做版本管理和监控。

Score candidate pairs on name similarity, shared domain, address, industry and shared employees; start with rules and move to a trained classifier once labels exist.

第 5 步 · 聚类（clustering）

要解决什么：A 和 B 像、B 和 C 像，那 A、B、C 是不是一家?

可选方案：

- 直接传递合并（transitive closure）：简单，但会链式乱合，最后把一大堆不相关的公司合成一团
- 受限聚类：限制簇大小、要求新成员和簇里多数成员都像

选哪个：受限聚类，加上超大簇报警。

为什么：链式乱合是实体消歧最常见的灾难，下游人数会突然暴涨。

代价 / 取舍：会漏掉一些真该合并的长链，交给人工处理。

I'd cluster with constraints, a size cap and majority agreement, because plain transitive closure chains unrelated companies together.

第 6 步 · 种子数据和 ID 管理

要解决什么：怎么让大公司先对？公司 ID 怎么保持稳定？

可选方案：

- 从零开始全靠算法
- 用官方注册库、上市公司名单、域名库当锚点（seed）

选哪个：先用种子把大公司钉死；每个公司 ID 带别名集合（所有叫过的名字和域名）和改名历史；并购记成母子关系；ID 永不复用、可退役。

为什么：大公司影响最大，用权威数据锚定最划算；稳定 ID 让下游的时间序列不断。

代价 / 取舍：种子数据要买或者维护。

Seed the big companies from official registries, keep alias sets and rename history per company id, record acquisitions as relationships, and never reuse ids.

第 7 步 · 质量监控和人工

要解决什么：上线后怎么知道没合错?

可选方案：

- 只看程序有没有报错
- 每天看指标 + 抽样 + 人工审核关键部分

选哪个：每天看：新 ID 比例、合并比例、最大簇大小、每家公司人数的日变化（大公司门槛更低）；分层抽样 + 判官；人工只审三类：超大簇、低置信合并、头部大公司。

为什么：错合会表现为某家公司人数突然暴涨，最容易从指标上看出来。

代价 / 取舍：人工有成本，所以只审最关键的。

I'd track new-id and merge rates, max cluster size and daily headcount changes per company, sample with judges, and send oversized clusters, low-confidence merges and top companies to human review.

第 8 步 · 总取舍：存疑时合还是分

要解决什么：和你的 take-home 正好相反。

可选方案：

- 存疑就合（take-home 的选择）
- 存疑就分

选哪个：存疑就分开，靠种子和人工来合。

为什么：take-home 里错合只丢一个头衔，漏合是一个重复 id 永远带着；这里错合会让两家公司人数加在一起，统计全错，代价反过来了。

代价 / 取舍：会有一些重复的公司 ID，需要人工慢慢合。

Here I'd split when in doubt, the opposite of the take-home, because a wrong merge corrupts headcounts while a missed merge only leaves a duplicate to fix later.

13 · 其他常被问的一页

- **负载均衡（load balancer）**：把请求分给多台机器（轮询 / 最少连接 / 按哈希）；本身要双活防单点。
- **限流**：令牌桶（token bucket）（匀速放令牌，有令牌才能过，允许短突发）；滑动窗口计数。
- **CDN**：静态内容放到离用户近的节点。
- **数据库索引（B-tree index）**：B 树，像书的目录；加速读、拖慢写；查什么就按什么建。
- **SQL vs NoSQL**：要事务和复杂查询用 SQL；要水平扩展、简单键值、灵活 schema 用 NoSQL。
- **分布式锁 / 单写者（distributed lock / single writer）**：同一时刻只允许一个人改，租约 + 过期防止持锁者挂掉。
- **隐私**：被遗忘权 → 删除名单在输出时过滤；日志不打描述原文。
- **容错（fault tolerance: retry, backoff, circuit breaker）**：重试 + 退避 + 幂等；超时；断路器（下游挂了就先别打）；单桶重试而不是整体重跑。

14 · 你的项目 ↔ 概念 速查

你做了的	概念名（答题时点名）
user_id % 64 分桶、同人同桶	按键分区 / 分片；取模扩容要搬家 → 一致性哈希
每人状态机、按日期一次一个快照	有状态处理、事件时间、不用未来信息
指纹跳过 57.5%	内容哈希去重
user_id << 16 | 序号	Snowflake 式分段 ID、可复现
规则 2 找候选 → 规则 5 打分 → 规则 6 折叠	Blocking → 成对比较 → 受限的传递合并（record linkage）
state-in / state-out、只处理有新数据的人	增量计算、检查点
迟到数据只重放一个人	事件时间 vs 处理时间、水位线后的更正
全量 = 半量 + 增量、跑两次一样	幂等、确定性
一个人出错回滚进隔离	故障隔离、部分失败不阻塞
先写临时文件再改名	原子写
cgroup CPU 配额、每进程 1 线程	容器资源感知、避免线程超订
Spark mapInArrow 跑同一函数	MapReduce / shuffle by key、Arrow 零拷贝
状态放 Iceberg 按 user_id 分区	数据湖表格式：ACID、MERGE、分区裁剪、时间旅行
run.json 指标、覆盖率、倒序日期 0	数据质量监控、无标注健康度
判官 + 人工 + 回归测试	离线评估、金丝雀

15 · 英文口述版 · 一句话库

- Partitioning: "I'd partition by user_id so all of one person's data lands on one worker; nothing is shared across people, so it scales linearly."
- Consistency: "It's a daily batch, so eventual consistency is fine as long as each day's output is published atomically."
- Queue: "A Kafka topic partitioned by user_id keeps one person's events in order; the consumer is idempotent so at-least-once delivery is safe."
- Batch vs stream: "The freshness requirement is daily, so batch wins; streaming would add ordering and state complexity with no benefit."
- Late data: "observed_ts is event time; when something arrives late I replay only that user, which is cheap and verifiable."
- Storage: "Parquet on object storage for the output; Iceberg for the state so I can merge only touched users and roll back a bad run."
- Monitoring: "Each run emits counts per rule; I'd alert on deviations and hard-fail on anything that must be zero."
- Trade-off line: "I chose X over Y because of Z; the cost is W, and here's how I'd notice if W becomes a problem."

名词详解（附录：网页里点名词会就地展开；PDF 看这里）

Spark

一个**分布式计算引擎**（distributed compute engine）：把一份大数据切成很多块，分给一群机器同时算，最后把结果合起来。

比方：一万张试卷，一个老师改要一个月；分给 100 个老师，每人 100 张，一天改完。Spark 就是那个分卷子、收卷子、谁请假了把他那叠重新分给别人的教务处。

- **driver（驱动程序）**：总指挥，一台机器，决定怎么切、谁算哪块。
- **executor（执行器）**：干活的进程，分布在很多机器上，每个处理几个分区（partition）。
- **DataFrame**：Spark 里的表，写法像 pandas/SQL，背后自动分布式执行。
- **惰性执行（lazy evaluation）**：你写的操作先记下来，最后要结果时才一起优化、一起跑。
- **shuffle（洗牌）**：按 key 重新分配数据，让同一个 key 的行到同一台机器。最贵的一步，因为要走网络。你的 repartition by user_id 就是一次 shuffle。
- **容错**：某台机器挂了，Spark 记得这块数据是怎么算出来的，换台机器重算这一块就行。

你的项目：mapInArrow 把每个分区以 Arrow 表交给 Python 函数（你的 coalesce_bucket），返回 Arrow 表。Spark 只管分发和收集，逻辑全在你的函数里。

Spark is a distributed engine: a driver splits the data into partitions, executors on many machines process them in parallel, and a shuffle regroups rows by key across the network. It's lazy, so it plans the whole job before running, and it recovers from a lost machine by recomputing just that partition.

MapReduce

分布式计算最早的模型（Google 2004），Spark 是它的升级版。

- **map**：每台机器各自处理自己那块数据，给每条输出打一个 key。比方：100 个老师各自改卷子，每张卷子贴上「班级」标签。
- **shuffle**：按 key 把东西送到一起，同一个班的卷子都送到同一个老师那。
- **reduce**：每个 key 的东西合并处理，算这个班的平均分。

缺点：每一步都把中间结果写硬盘，慢。Spark 尽量放内存，所以快很多。

MapReduce is map each partition, shuffle by key, then reduce each key's group. Spark follows the same shape but keeps intermediate data in memory.

Kafka

一个**分布式消息队列 / 日志**（distributed log）：生产者往里写消息，消费者按自己的速度读，消息存几天不丢，可以倒回去重读。

比方：快递站的传送带。寄件人把包裹放上去就走；分拣员按自己的速度拿；传送带上的包裹存一周，分拣员请假回来可以从上次的位置接着拿。

- **topic（主题）**：一类消息，比如「新抓到的档案」。
- **partition（分区）**：一个 topic 切成多条传送带，并行。每条只往末尾追加。
- **offset（偏移量）**：消息在分区里的位置号，递增。消费者记住自己读到第几号。
- **broker**：存消息的服务器，一个集群有很多台，每个分区有几份副本（replica）防丢。
- **producer / consumer（生产者 / 消费者）**；**consumer group（消费者组）**：一组消费者分摊一个 topic 的分区。
- **保留期（retention）**：消息存多久，比如 7 天，过期删掉。

它不是什么：Kafka 不做计算，只负责「传送和暂存」。计算交给 Spark / Flink。

Kafka is a distributed append-only log. Producers write to topics split into partitions, each message gets an offset, and consumers track their own offsets, so they can read at their own pace and replay. It moves and stores data; computation happens in Spark or Flink.

Flink

一个**流处理引擎**（stream processing engine）：数据一条一条来，一来就算，结果秒级更新。和 Spark（攒一批再算）相对。

比方：Spark 是每天晚上结一次账；Flink 是收银台每笔交易当场入账。

- **状态（state）**：流处理要记住之前的东西（比如这个人目前有哪些职位），Flink 帮你存、帮你做检查点（checkpoint）防丢。
- **事件时间与水位线（event time & watermark）**：处理乱序和迟到。
- **代价**：要一直开着跑、状态管理复杂、出错难排查，运维（operations）比批处理重得多。

Flink processes events one by one as they arrive, keeping per-key state with checkpoints, so results update in seconds. The cost is a long-running job with harder state management and operations than a daily batch.

Kafka Streams

Kafka 自带的一个**轻量流处理库**：直接在你的 Java 程序里读 Kafka、算、写回 Kafka，不需要单独的集群。比 Flink 简单，适合中小规模。

Kafka Streams is a library for stream processing inside your own app, reading from and writing back to Kafka without a separate cluster.

Arrow

一种**内存里摆放表格数据的标准格式**，按列摆（columnar in-memory format）。

比方：标准集装箱。Polars、Spark、pandas、DuckDB 都认这种箱子，交接数据时不用拆箱重装，叫**零拷贝（zero-copy）**。

你的项目：Spark 把每个分区以 Arrow 表交给 Polars，你的函数处理完再以 Arrow 表交回，中间不做转换，所以单机和 Spark 能跑同一个函数。

Arrow is a standard columnar layout for data in memory, so Spark, Polars and pandas can hand tables to each other without copying or converting.

Parquet

一种**列式文件格式**（columnar file format），存在硬盘 / S3 上。

- **按列存**：同一列的值放在一起。只要 3 列就只读 3 列（列裁剪，column pruning）。
- **压缩好**：同一列类型一样、值相似，压缩率高。
- **带统计信息**：每块记最大最小值，查「日期 > 2025」时整块能跳过（谓词下推，predicate pushdown）。
- **不能改**：要改一行只能重写整个文件。

Parquet stores data by column with compression and per-block min-max statistics, so a query reads only the columns it needs and skips blocks that can't match. Files are immutable; changing a row means rewriting the file.

Iceberg（表格式）

一种**数据湖表格式**（table format）：在一堆 parquet 文件上面加一层「目录 / 清单」，让它们像一张数据库表一样能事务提交、回退、按键更新。同类：Delta Lake、Hudi。

- **ACID 提交**：写新文件 → 写新清单 → 原子地把指针换到新清单。读的人永远看到完整的一版。
- **快照 / 时间旅行（snapshot / time travel）**：每次提交是一个版本，能回到任何一版。
- **MERGE INTO**：按键更新，只改动过的行（底层是写新文件 + 标记旧的作废）。
- **分区（partitioning）**：按某列分目录，查询只读相关分区。
- **catalog（目录服务）**：存「每张表当前指向哪份清单」的地方，比如 AWS Glue、Hive Metastore。

Iceberg is a table format over parquet files: it tracks which files make up each version, commits atomically by swapping a pointer in the catalog, supports MERGE, partition pruning and time travel.

S3 / 对象存储

AWS 的**对象存储**（object storage）：一个几乎无限大的网盘，按「桶 + 路径」存文件。

- 便宜、自动多副本、几乎不会丢。
- 只能整文件读写，不能改文件中间的一行，也不能追加。
- 一次读几十毫秒，适合批量读大文件，不适合频繁点查。

S3 is object storage: cheap, durable, replicated, whole-file reads and writes, good for batch data and bad for small random updates.

KV 存储：DynamoDB / Cassandra / Redis

键值存储（key-value store）：给一个 key 拿一个 value，毫秒级。

- **DynamoDB**：AWS 托管，不用自己管机器，按分区键（partition key）哈希分片。
- **Cassandra**：开源，自己部署，多副本、写入很快、最终一致。
- **Redis**：数据放内存，最快（微秒级），常用作缓存。
- 共同缺点：不擅长复杂查询、join、按别的字段查。

Key-value stores like DynamoDB, Cassandra and Redis answer get-by-key in milliseconds by hashing the key to a partition. They're fast for point lookups and poor at joins or queries on other fields.

Redis

把数据放在**内存**里的键值存储，读写微秒级，最常用作**缓存**（cache）。也能做计数器、排行榜、分布式锁。数据主要在内存，断电要靠持久化配置才不丢。

Redis is an in-memory key-value store, usually used as a cache, with microsecond reads.

数据仓库 / 查询引擎：BigQuery / Snowflake / Trino

给分析师写 SQL 查大表用的系统（data warehouse / query engine）。

- **BigQuery、Snowflake**：云上托管的数仓，存储和计算都帮你管。
- **Trino**：开源查询引擎，直接查 S3 上的 parquet / Iceberg 表，不搬数据。
- 擅长：整表扫描、聚合、join。不擅长：毫秒级点查。

Warehouses and query engines run SQL over large tables for analysis; they're good at scans and aggregations, not millisecond lookups.

B 树索引

数据库最常用的**索引结构**（B-tree index）：一棵很矮很胖的排序树，像书的目录，几次跳转就能找到任何一行。

- 数据按 key 排好序存在一页一页里（每页几 KB）。
- key 递增插入：总是写最后一页，很快。
- key 随机插入：要找到中间某页插进去，页满了要拆成两页（**页分裂，page split**），慢，而且页都半空，占空间。这就是 UUID 当主键慢的原因。

A B-tree keeps keys sorted in pages so lookups take a few hops. Increasing keys append to the last page; random keys like UUIDs insert in the middle and cause page splits.

哈希（hash）

把任意内容变成一个固定长度数字的函数（hash function）。同样的输入永远得到同样的数字；输入差一点，数字就完全不同。

用处：分区（hash(key) % N 决定去哪台机器）、去重（内容一样哈希就一样，你的指纹）、快速比较（比数字比比整段文字快）。

撞号（collision）：两个不同输入得到同一个数，64 位哈希概率极低。

A hash function maps any input to a fixed-size number deterministically; it's used to route keys to partitions, detect duplicates and compare content quickly.

QPS / 延迟 / 吞吐

- **QPS（queries per second）**：每秒多少个请求。
- **延迟（latency）**：一个请求从发出到拿到结果用多久。常看 p99：最慢的那 1% 有多慢。
- **吞吐（throughput）**：单位时间处理多少数据，批处理更看这个。

比方：高速收费站。QPS = 每秒过几辆车；延迟 = 一辆车排队加缴费用多久；吞吐 = 一小时总共过多少车。

QPS is requests per second, latency is how long one request takes, often measured at p99, and throughput is how much work gets done per unit of time.

复制：主从（leader–follower）

同一份数据存在好几台机器上（replication）。

- **主（leader）**：只有它能写。
- **从（follower / replica）**：从主那里抄改动，可以分担读。
- **复制延迟（replication lag）**：从库抄得慢一点，刚写完去从库读可能读到旧值。
- **故障切换（failover）**：主挂了，选一个从升为主。

In leader–follower replication one node takes writes and followers copy them; reads can go to followers but may be slightly stale, and a follower is promoted if the leader fails.

缓存（cache）和 cache-aside

缓存 = 把常用数据放在更快的地方（通常是内存，比如 Redis），省得每次去慢的数据库拿。

cache-aside（旁路缓存），最常用的用法，三步：

1. 读：先查缓存，有就直接返回（命中，hit）。
2. 没有（未命中，miss）：去数据库查，查到后写进缓存再返回。
3. 写数据库时：把缓存里的旧值删掉，下次读时重新加载。

TTL（time to live）：每个缓存条目的存活时间，到点自动删除。

With cache-aside, the app reads the cache first, falls back to the database on a miss and fills the cache, and deletes the cached value when the data changes. Each entry has a TTL.

淘汰策略：LRU / LFU

缓存满了要删一些，删谁?

- **LRU（least recently used）**：删最久没被访问的。像衣柜里一年没穿的衣服先扔。
- **LFU（least frequently used）**：删访问次数最少的。

你的代码里 Python 的 `lru_cache` 就是 LRU：缓存 title_key 等函数的结果，满了扔最久没用的。

LRU evicts the entry unused for the longest time; LFU evicts the one used least often.

Snowflake ID

Twitter 发明的**分布式唯一 ID**方案（不是 Snowflake 数据仓库那家公司）。

64 位拆成几段：1 位符号 + 41 位毫秒时间戳 + 10 位机器号 + 12 位序号（同一毫秒内的第几个）。每台机器只用自己的机器号发号，互不冲突，不用中心协调；高位是时间所以大致按时间递增。

你的项目：同样「拆段各管各的」，但用 user_id + 每人序号，不放时间戳，因为要可复现。

A Snowflake id packs a timestamp, a machine id and a per-millisecond sequence into 64 bits, so every machine can issue unique, roughly time-ordered ids without coordination.

UUID

128 位的随机唯一号，例如 `550e8400-e29b-41d4-a716-446655440000`。谁都能自己生成、几乎不会撞。缺点：长一倍、随机无序（B 树插入慢）、不含时间信息。新版 UUID v7 前面带时间戳，变成有序。

A UUID is a 128-bit random identifier anyone can generate without coordination; it's large and unordered, which hurts index performance, unless you use time-ordered UUID v7.

CAP 定理

分布式系统的一个定理：**C 一致性（consistency）**所有机器读到一样的值；**A 可用性（availability）**任何时候请求都有回应；**P 分区容忍（partition tolerance）**机器之间网络断了还能工作。网络一定会偶尔断，所以 P 必须要，实际是在断网那段时间选 C 还是 A。

比方：两家分店电话线断了，要么停业等恢复（保 C），要么照常卖之后再对账（保 A）。

When the network splits, a distributed system has to choose between consistency and availability; partition tolerance isn't optional.

幂等（idempotent）

同一个操作做一次和做十次，结果一样。比方：电梯按钮按一下和按十下都一样；但「账户加 100 元」不是幂等的，做两次就多加了。

为什么重要：分布式系统里网络超时了你不知道对方收没收到，只能重试；只有幂等的操作才能放心重试。

怎么做到：每个操作带唯一 id，处理过的记下来，再来就跳过；或者把「加 100」改成「设成 X」。

An idempotent operation has the same effect no matter how many times it runs, which is what makes retries safe.

投递语义（delivery semantics）

- **至多一次（at-most-once）**：发了就不管，可能丢，不会重。
- **至少一次（at-least-once）**：没确认就重发，不会丢，可能重复。
- **恰好一次（exactly-once）**：不丢不重，最贵。

实际最常用：至少一次 + 消费端幂等 = 效果上的恰好一次。比方：快递没签收就再送一次，收件人看到重复的包裹编号就拒收第二个。

Most systems use at-least-once delivery with an idempotent consumer, which gives effectively-once results without the cost of true exactly-once.

事件时间 / 处理时间 / 水位线

- **事件时间（event time）**：事情真正发生的时间。你的 observed_ts（抓取日期）。
- **处理时间（processing time）**：系统收到 / 处理它的时间。你的「数据到达那天」。
- 两者之差就是迟到。
- **水位线（watermark）**：系统宣布「X 时刻之前的事件应该都到齐了」，可以出结果了。水位线之后才到的旧事件 = 迟到数据，要么丢、要么回头更正已出的结果。

比方：老师定周五 5 点收作业（水位线），5 点后交的要么不收，要么改完再补分。

Event time is when something happened, processing time is when the system sees it, and a watermark declares that everything before a given time has arrived; anything older after that is late.

Lambda / Kappa 架构

- **Lambda**：两套系统并行，流处理给秒级的近似结果，批处理晚上算准确结果覆盖掉。缺点：同一套逻辑写两遍。
- **Kappa**：只要流处理，要重算就从 Kafka 开头重放一遍。

你的情况：只要天级，一套批就够，两者都不需要。

Lambda runs a fast streaming path and a correct batch path side by side; Kappa keeps only streaming and replays the log to recompute.

数据倾斜（data skew）

某个 key 的数据特别多，分到它的那台机器要干一整晚，别的机器早干完在等。比方：100 个老师改卷，一个老师分到 5000 张，其他人每人 50 张。

办法：加盐（salting）把大 key 拆成几份分给几台机器；两阶段聚合（先局部合并再全局合并）；大 key 单独处理。

你的数据：每人平均 29 条、最多 166 条，没有倾斜。

Skew is when one key has far more data than others, so one worker becomes the bottleneck; salting or two-phase aggregation spreads it out.

cgroup

Linux 用来**限制一组进程能用多少资源**的机制（control groups）。Docker 的 --cpus 4、--memory 16g 底层就是 cgroup。

坑：程序里调 os.cpu_count() 看到的是整台机器的核数（比如 14），不是容器被限的 4 核，开 14 个进程就超了。要读 /sys/fs/cgroup/cpu.max 里的配额。

cgroups are how Linux limits a container's CPU and memory; os.cpu_count reports the host, so you read the cgroup quota to size the worker pool.

Docker

把程序和它需要的所有依赖打包成一个**镜像**（image），在任何机器上都能一模一样地运行。运行起来的叫**容器**（container）。比方：搬家时把整个房间连家具一起装进集装箱，到哪都一样。

Docker packages an app and all its dependencies into an image that runs identically anywhere as a container.

负载均衡（load balancer）

站在一群服务器前面，把进来的请求分给它们。比方：银行大堂经理把客户分到各个窗口。

分法：轮询（round robin，挨个分）、最少连接（谁最闲给谁）、按哈希（同一用户总去同一台）。它自己也要有备份，否则它一挂全挂。

A load balancer spreads incoming requests across servers by round robin, least connections or hashing.

限流：令牌桶（token bucket）

一个桶，每秒匀速往里放 N 个令牌，桶最多装 M 个。每个请求要拿一个令牌才能通过，没令牌就拒绝或排队。效果：平均每秒最多 N 个，允许短时间突发 M 个。

比方：游乐园每分钟放 10 张票，攒着的票最多 50 张。

A token bucket refills at a fixed rate up to a cap; each request takes a token, so it allows short bursts but limits the average rate.

断路器（circuit breaker）/ 重试退避（retry with backoff）

- **重试退避**：失败了等 1 秒重试，再失败等 2 秒、4 秒……，越等越久，别把对方打死。
- **断路器**：下游连续失败太多次，就先「断开」一段时间，直接返回失败不再调用，过一会儿试探一下恢复了再接上。比方：家里保险丝，短路了先跳闸，别把整栋楼烧了。

Retries back off exponentially, and a circuit breaker stops calling a failing dependency for a while so it can recover.

Blocking（候选生成）

去重 / 实体消歧里的第一步：先用便宜的条件把「可能是同一个」的候选缩小，再对候选做贵的比较。

为什么必须：N 条两两比是 N^2 次。1000 万条就是 10^{14} 次，算不完。按「同雇主」「同域名」分组后，每组只有几十条。

你的项目：规则 2 找候选（同雇主 + 日期兼容）就是 blocking。

Blocking narrows comparisons to plausible candidates with a cheap key, so you avoid comparing every pair.

B-cubed（簇指标）

评估「分组 / 聚类」好坏的指标。对每一条记录，看它所在的簇里：有多少比例真的和它是一组（精确率），它真正的组员有多少比例被分到了一起（召回）。所有记录取平均。

和成对指标的区别：成对指标看每一对配没配对；B-cubed 按记录算，大簇里的错误不会被放大。

B-cubed scores each record by how much of its cluster truly belongs with it and how much of its true group it was clustered with, then averages.

embedding / 向量最近邻

用模型把一段文字变成一串数字（比如 384 个数），意思相近的文字数字也相近。「Software Engineer」和「软件工程师」的向量会很接近。

最近邻（nearest neighbour）：给一个新头衔的向量，在库里找最接近的那几个。**向量库**（vector database，如 FAISS）专门做这个，毫秒级。

An embedding turns text into a vector so similar meanings are close together; a vector index finds the nearest ones quickly.

CDN

内容分发网络（content delivery network）：把图片、视频这类静态文件复制到全世界很多节点，用户从最近的节点拿。比方：总部的书在各地都开分店。

A CDN caches static content at edge locations close to users.

SQL vs NoSQL

- **SQL（关系型数据库，PostgreSQL / MySQL）**：表结构固定、支持事务和 join、强一致。扩展主要靠一台更强的机器。
- **NoSQL（DynamoDB / Cassandra / MongoDB）**：结构灵活、天生分布式、容易水平扩展（加机器），通常最终一致、不支持复杂 join。

Relational databases give schemas, joins and transactions; NoSQL stores trade those for easy horizontal scaling.

分布式锁 / 租约（lease）

多台机器同时想改同一份数据，用一把锁保证同一时刻只有一个人改。锁带**租约**（过期时间），持锁的机器挂了，锁到期自动释放，不会永远卡住。

A distributed lock lets only one worker modify something at a time, with a lease so a crashed holder doesn't block forever.

血缘（lineage）

记录每份数据「从哪来、经过哪些步骤」。比方：食品追溯码，扫一下知道哪个农场、哪个工厂。出了错能顺着找到是哪一步、哪份输入的问题，也知道改了上游会影响哪些下游。

Lineage records where each dataset came from and which steps produced it.

数据契约（data contract）

上游和下游之间的「约定」：有哪些列、什么类型、什么含义、多久更新一次。上游要改必须提前通知。你的 conform「缺列直接报错」就是在执行契约：不符合就停，不悄悄出错数据。

A data contract fixes the schema and meaning a producer promises to consumers, and changes must be announced.

scatter-gather

把一个查询同时发给所有分区（scatter，撒出去），每个分区各自查，再把结果收回来合并（gather）。数据按 user_id 分区，却要按公司查时，只能这么做，分区越多越慢。

Scatter-gather sends a query to every partition and merges the answers, used when the query key isn't the partition key.

ACID

事务的四个保证：**A 原子性（atomicity）**要么全成功要么全不算；**C 一致性（consistency）**不破坏规则；**I 隔离性（isolation）**并发的事务互不干扰；**D 持久性（durability）**提交了就不会丢。

ACID means a transaction is all-or-nothing, keeps the data valid, is isolated from concurrent ones, and survives crashes once committed.

检查点（checkpoint）

定期把当前的进度 / 状态存下来，挂了从最近的存档接着跑，不用从头。比方：游戏存档。你的 --state-out 就是每天一个检查点；bucket.py 里处理每个人前 deepcopy 一份状态，也是一个小检查点（出错回滚）。

A checkpoint saves progress so a job can resume from there instead of starting over.